

Modbus Register Map — 303Modbus13

Modbus RTU protocol documentation for the **Ideaflex 303Modbus13** relay module, aligned with **303Modbus13 RTU Control** (`app.py`, `modbus_device.py`).

Hardware variant supported by the application

Item	Application value	Code constant
Relays	2 (CH1, CH2)	RELAY_COUNT = 2, coil addresses 0–1
Opto inputs	2 (IN1, IN2)	INPUT_COUNT = 2, discrete input addresses 0–1
Default port	COM4	SerialSettings.port
Default Slave ID	0xFF (255)	FACTORY_SLAVE_ID = 255
Library	pymodbus 3.8.6	requirements.txt

Module firmware may still expose addresses 0–7 (up to 8 channels). The GUI controls and displays only the **first 2** relays and **first 2** inputs. The advanced console can send any frame (e.g. `count=8`).

Communication parameters

Parameter	Default
Protocol	Modbus RTU
Port (app)	COM4 (USB-RS485 converter, e.g. CH340)
Baud rate	9600 (scan: 9600, 4800, 19200)
Format	8N1 (8 data bits, no parity, 1 stop bit)
Slave ID (relays)	0xFF / 255 (factory), then configurable (e.g. 0x01)
Slave ID (T/H sensor)	0x02 (optional)

Note: The module can be connected directly via RS-485 (COM port) or through a Modbus TCP/RTU gateway (e.g. USB-M0). The register map is identical in both cases.

Timeouts and reliability (recommended values)

With a USB-RS485 converter (e.g. CH340) at 9600 baud the module responds correctly, but replies can be delayed or unstable with a short timeout. The reference application uses:

Parameter	Value	Notes
Serial timeout	3.0 s	Full Modbus response wait (pymodbus: timeout=3.0)
Retries (pymodbus)	3	Retry on transport error
Post coil write delay	0.35 s	Delay before next read after <code>write_coil</code> / <code>write_coils</code>
Read retries	3×	Extra <code>read_coils</code> / <code>read_discrete_inputs</code> attempts with 0.15–0.2 s pause
Connect (ping)	3 attempts	Read coils → discrete inputs → test write; 0.2–0.6 s between attempts
Port scan	0.4 s timeout, 1 retry	Fast module discovery on the bus

Client initialization example (pymodbus):

```
from pymodbus.client import ModbusSerialClient client = ModbusSerialClient( port="COM4",  
baudrate=9600, parity="N", stopbits=1, bytesize=8, timeout=3.0, # do not use 0.5–1.0 s –  
frequent timeouts on new modules retries=3, )
```

Practical notes:

- **Coil write (FC 0x05 / 0x0F)** on modules with Slave ID 0xFF is more stable than coil read (FC 0x01) — if read fails, use cached state after a successful write.
- **Opto input read (FC 0x02)** requires `count=8` in the frame (as in the original app), even when the module has only 2 physical inputs — interpret result as IN1, IN2, ...
- For custom integration (PLC, script), use **timeout ≥ 3 s** and **at least 2–3 retries** before reporting a communication error.
- **FC 0x10** (Write Multiple Registers) responses may be non-standard — the app applies a decoder patch in `modbus_device.py`.

Application behavior (303Modbus13 RTU Control)

Operation	Implementation
Connect (ping)	Sequence: <code>read_coils(0,8)</code> → <code>read_discrete_inputs(0,8)</code> → <code>write_coil(0,False)</code> ; 3 attempts, 0.2–0.6 s pause

Single relay control	<code>write_coil</code> → 0.35 s pause → <code>read_discrete_inputs</code> (<code>write_coil_and_read_inputs</code>)
Both ON/OFF	<code>write_coils(0, [state] * 8)</code> — 8 coils in frame (as in <code>apka_4.py</code>)
Relay read in GUI	<code>read_coils(0, 2)</code> ; on error — cache of last successful write (<code>relay_cache</code>)
Input read	<code>read_discrete_inputs(0, count=8)</code> → result IN1 , IN2 (first 2 bits)
Auto-refresh	Every 1 s (<code>_poll_interval_ms = 1000</code>), off by default
Module scan	COM ports × baud [<code>9600</code> , <code>4800</code> , <code>19200</code>] × slave [<code>255</code> , <code>1</code> , <code>0</code> , <code>2</code> , <code>3</code> , <code>4</code> , <code>5</code>]; timeout 0.4 s
Address change	FC <code>0x10</code> , register 0, Slave=0 (broadcast); reconnect with new ID after change

Relay Slave ID — main module

Communication with relays and opto inputs uses the address configured in the module (factory **0xFF**, after setup e.g. **0x01**). In examples below, `slave=255` is the factory address; substitute your Slave ID after reconfiguration.

Coils — relay outputs

Control relays CH1, CH2 (application). Firmware map may expose addresses 0–7 (8-channel variant).

Address	Name	Description	Read function	Write function
0	CH1	Relay channel 1	FC <code>0x01</code> Read Coils	FC <code>0x05</code> Write Single Coil
1	CH2	Relay channel 2	FC <code>0x01</code> Read Coils	FC <code>0x05</code> Write Single Coil
2–7	CH3–CH8	Optional (8-channel variant)	FC <code>0x01</code>	FC <code>0x05</code> / <code>0x0F</code>

Write multiple coils (app, Both ON/OFF): FC `0x0F`, address 0, count 8, values [`ON/OFF`] × 8 (firmware expects full frame as in `apka_4.py`).

Value	Meaning
-------	---------

0 / False	Relay off (OFF)
1 / True	Relay on (ON)

Example — turn CH1 ON (factory module, Slave 255):

FC 0x05, Slave=255, Address=0, Value=0xFF00 (ON)

Example — read relays (manufacturer documentation frame):

FC 0x01, Slave=255, Address=0, Count=8 → data byte: bit0 = CH1, bit1 = CH2, ...

Example — turn both relays ON (app, Both ON):

FC 0x0F, Slave=255, Address=0, Count=8, Values=[ON, ON, ON, ON, ON, ON, ON, ON]

Discrete inputs — opto inputs

Read opto input state IN1–IN8 (2 or 8 physical channels depending on module variant). Read-only — Modbus function **FC 0x02 Read Discrete Inputs**.

Address	Name	Description	Function
0	IN1	Opto input 1	FC 0x02 Read Discrete Inputs
1	IN2	Opto input 2	FC 0x02 Read Discrete Inputs
2	IN3	Opto input 3	FC 0x02 Read Discrete Inputs
3	IN4	Opto input 4	FC 0x02 Read Discrete Inputs
4	IN5	Opto input 5	FC 0x02 Read Discrete Inputs
5	IN6	Opto input 6	FC 0x02 Read Discrete Inputs
6	IN7	Opto input 7	FC 0x02 Read Discrete Inputs
7	IN8	Opto input 8	FC 0x02 Read Discrete Inputs

Example — read inputs (count=8 required in frame, as in the app):

FC 0x02, Slave=255, Address=0, Count=8 → GUI interpretation: bits[0]=IN1, bits[1]=IN2

Value	Meaning
0 / False	Input inactive (no signal on INx relative to COM)
1 / True	Input active (voltage in activation range on INx relative to COM)

Opto-isolated input wiring (IN / COM)

Inputs are **opto-isolated** — the signal circuit is **galvanically separated** from module power and logic ground. Current must flow through the optocoupler. This requires **two wires**: common **COM** and signal on **INx**.

Terminal	Role
COM	Common return of the input circuit (signal reference)
IN1 ... IN8	Signal input (voltage relative to COM)

Typical wiring (positive logic, PNP):

Signal supply (+12...24 V DC) (+) ■■■■■ switch / contact / sensor ■■■■■ INx (-) GND
■■■■■ COM

- Connect **COM** to **GND (0 V)** of the signal source (e.g. minus of 24 V supply).
- Connect **INx** to the **positive signal** (+12...24 V DC) — via relay contact, sensor output, etc.
- Activation voltage range: typically about **6–24 V DC** (see module datasheet).

Important — input COM ≠ board GND

Input terminal **COM** is **not** automatically tied to module power GND or RS-485 GND. It is a **separate, isolated** input circuit.

Voltage on INx alone without COM connected to signal GND will not close the circuit — the optocoupler stays off and Modbus always returns `False`, even if voltage is visible on a relay contact. Tie module power GND and input signal GND **only** when you intentionally share the same supply and the design allows it.

Wrong vs correct wiring:

WRONG (no current through opto): +24V ■■■ relay contact ■■■ IN1 (COM not connected → no current → always False in Modbus) CORRECT: +24V ■■■ contact ■■■ IN1 GND ■■■■■ COM

Service test: connect COM to supply GND, briefly apply +12...24 V to IN1 — in the app (FC 0x02) IN1 should show `True`.

Holding registers — configuration

Address	Name	Description	Read function	Write function
0	DEVICE_ADDR	Modbus slave address	FC 0x03 Read Holding Registers	FC 0x10 Write Multiple Registers

Change Modbus slave address

To change the device address, write the new address to holding register 0 with **Slave ID = 0** (broadcast):

```
FC 0x10, Slave=0 (broadcast), Address=0, Count=1, Values=[new_address]
```

Parameter	Range
new_address	0x01 – 0xFF

After changing the address, communicate with the module using the new Slave ID.

Factory address: A new module defaults to Slave ID **0xFF (255)**. Read it via FC 0x03, holding register 0. Set a new address (e.g. 0x01) with FC 0x10 and Slave = 0 (broadcast) — **only with a single module on the bus**. Then use the new address.

Frames from manufacturer portal (RTU):

Operation	Request frame (hex)	Response (hex)
Read address	00 03 00 00 00 01 + CRC	00 03 02 00 FF + CRC → address = 0xFF
Set address to 1	00 10 00 00 00 01 02 00 01 + CRC	acknowledge
Set address to 40	00 10 00 00 00 01 02 00 28 + CRC	acknowledge
Read relays	FF 01 00 00 00 08 + CRC	data byte: bit0 = R1, bit1 = R2

Slave ID = 2 — temperature and humidity sensor (optional)

The module may include a T/H sensor. If not installed, requests to Slave ID = 2 will not get a response.

Input registers — measurements

Address	Name	Description	Function	Unit
1	TEMPERATURE	Temperature	FC 0x04 Read Input Registers	°C × 10

2	HUMIDITY	Relative humidity	FC 0x04 Read Input Registers	% × 10
---	----------	-------------------	------------------------------	--------

Value conversion:

```
temperature_C = register[1] / 10.0 humidity_% = register[2] / 10.0
```

Example — read temperature and humidity:

```
FC 0x04, Slave=2, Address=1, Count=1 → temperature FC 0x04, Slave=2, Address=2, Count=1 → humidity
```

Example response:

```
Register TEMPERATURE = 235 → 23.5 °C Register HUMIDITY = 582 → 58.2 %
```

Modbus function summary

FC code	Name	Use in 303Modbus13 (app)
0x01	Read Coils	Read CH1–CH2 (GUI: count=2; ping: count=8)
0x02	Read Discrete Inputs	Read IN1–IN2 (count=8 in frame, 2 result bits)
0x03	Read Holding Registers	Read configuration (device address)
0x04	Read Input Registers	Read T/H (Slave ID = 2)
0x05	Write Single Coil	Turn CH1 or CH2 on/off
0x06	Write Single Register	Write single holding register
0x0F	Write Multiple Coils	Both ON/OFF (count=8 in frame)
0x10	Write Multiple Registers	Change Modbus address (broadcast, Slave=0)

RS-485 wiring diagram

```
[PC / PLC] [303Modbus13] | | USB-RS485 ■■■ A/B (RS-485) ■■■ A/B | | COM Power
```

- Line **A** → **A**, line **B** → **B**

- Common GND recommended
- Maximum bus length and termination per module documentation

Troubleshooting

Problem	Possible cause	Solution
No Modbus response	Wrong COM port or baud rate	Check 9600 8N1, correct COM port
No Modbus response	Wrong Slave ID	New module: 0xFF; older: 0x01; change address via broadcast (Slave=0)
Timeout despite correct wiring	Master timeout too short	Set timeout ≥ 3 s, retries ≥ 3
Relay write OK, coil read unreliable	Firmware limitation / FF address	Use known state after write; ping via <code>write_coil</code>
Opto inputs always <code>False</code>	No IN–COM circuit	Wire COM to GND and INx to +signal (opto isolation — see above)
Opto inputs always <code>False</code>	Voltage on contact only, not on IN terminals	Signal must reach module INx and COM terminals
T/H sensor no response	Sensor not installed	Normal — module works without sensor
CRC / timeout error	RS-485 wiring issue	Check A/B polarity, bus GND connection

Contact

If you have any problems or questions, contact **FlexTech_KRK** on [Allegro](#) or email biuro@ideaflex.pl.

© Ideaflex sp. z o.o. — documentation for 303Modbus13 module